

An Adaptive-Difficulty Layer for the Bowling Machine

Making the machine adjust to the player, and level up with them

A companion module to “The Physics of a Cricket Ball in Flight” — that report models what one ball does in the air; this one decides which ball to bowl next, based on how the player has been doing.

12 September 2026

The trajectory engine and machine-control mapping answer a fixed question: given these release parameters, what does the ball do, and what wheel RPMs produce them? This module answers a different, ongoing one: given how this specific player has been performing, what should the machine bowl next — and how should that change as they get better? That's the “adopt the player's capacity and upgrade with expertise” idea, turned into an algorithm.

1. The idea: aim for the challenge point, not the hardest ball

Coaching science, sports psychology, and game design all converge on the same finding under different names — the “challenge point”, the “flow channel”, Vygotsky's “zone of proximal development”. A learner improves fastest against a challenge that is hard enough to stretch them, but not so hard that success becomes essentially random. Bowl the easiest ball every time and there's nothing to learn from; bowl the hardest ball every time and practice becomes a lottery. The target is a delivery just above the player's current level — which means the machine needs two things on the same scale: a running estimate of how good the player currently is, and a real difficulty score for any delivery it could bowl.

This is a design choice, not a law of physics — the challenge-point target and how aggressively to chase it (the “margin” below) are coaching parameters you can tune, the same way you'd adjust a training plan.

2. Rating both sides on one scale: player vs. delivery

The mechanism reused here is Elo — the rating system chess uses to rate a player against an opponent, extended so one “player” in the match is a batter and the other is a cricket delivery. Given a player rating and a delivery rating, Elo gives a clean probability that the player comes out on top:

$$E_{player} = \frac{1}{1 + 10^{(R_{delivery} - R_{player})/400}}$$

After the ball is faced, someone reports an outcome — a coach or the player themselves for now (no extra hardware needed), or in principle an automated vision/sensor system later, since the code doesn't care where the number comes from. That outcome is scored 0 (complete failure) to 1 (complete success), and the player's rating moves by the standard Elo update rule:

$$R_{player} \leftarrow R_{player} + K(S_{outcome} - E_{player})$$

K controls how fast the rating reacts — higher K means the machine adapts quickly to a hot or cold run, lower K means it trusts the long-run average more. A starting point of K=24 is used here; it's a coaching dial, not a fitted constant.

Outcome	Score	Meaning
missed	0.00	played and missed entirely / bowled
beaten	0.05	beaten by swing/seam/pace, no meaningful bat contact
edged	0.30	thick outside edge, risky contact
defended	0.55	solid defensive shot, no attacking intent
controlled	0.75	attacking shot, kept along the ground under control
boundary	0.95	four
six	1.00	six

Starting scores to tune, not gospel — a coach may want “defended” worth more against a genuine yorker than against a long hop, for instance. The module takes either one of these labels or a raw 0-1 float, so a finer-grained scoring scheme drops straight in.

3. Delivery difficulty, derived from the actual physics

The delivery side of the match isn't guessed — it's computed from the same coefficient curves (Cd, Cl, Cs, Cds) that drive the trajectory simulator, so a delivery that is genuinely nastier in the physics automatically rates as harder here too. Four forces, each turned into a dimensionless “force relative to the ball's own weight” factor so they're comparable, are combined into one 0-1 “nastiness” score:

$$N = 0.45 f_{pace} + 0.20 f_{swing} + 0.10 f_{seamdrag} + 0.25 f_{spin}$$

- **Pace factor** — raw release speed relative to a ~160 km/h practical ceiling: less reaction time for the batter, weighted highest.
- **Swing factor** — peak seam-swing force (Section 3.4 of the main report) relative to ball weight.
- **Seam-drag factor** — the extra late skid/deviation off a cross-seam ball, weighted lowest since it unsettles timing more than it threatens the stumps outright.
- **Spin factor** — Magnus force from the delivery's spin rate, relative to ball weight.

That nastiness score is then mapped onto an Elo-like scale centred on a base rating (1200) with a spread of ± 600 , so it composes directly with the player's own rating:

$$R_{delivery} = R_0 + (N - 0.5) \cdot 2 \Delta R$$

4. Choosing what to bowl next

With both ratings on the same scale, picking the next delivery becomes a search problem. The target is the player's current rating plus a small positive challenge margin (a coaching dial, default +60):

$$R_{target} = R_{player} + \text{margin}$$

The algorithm generates a pool of physically varied candidate deliveries across the machine's realistic speed/seam/spin range, rates each one with the same difficulty formula, and picks — with a touch of weighted randomness among the closest handful, not always the single nearest — whichever candidate sits closest to that target. The randomness matters: always serving the single mathematically “optimal” ball would make every over identical, which defeats the point of a training tool.

Pseudocode

```
FUNCTION suggest_next_delivery(player, ball, margin=60):
    target <- player.rating + margin
    candidates <- []
    REPEAT pool_size TIMES:
        d <- random_delivery(speed, seam_angle, spin_type, spin_rate)
        rating <- delivery_difficulty_rating(d, ball)
        candidates.append(|rating - target|, d)
    shortlist <- candidates sorted by gap, take closest 5
    RETURN weighted_random_choice(shortlist, weight = 1 / (1 + gap))

FUNCTION record_outcome(player, delivery, ball, outcome):
    score <- OUTCOME_SCORES[outcome] # or a raw 0-1 value
    d_rating <- delivery_difficulty_rating(delivery, ball)
    expected <- elo_expected(player.rating, d_rating)
    player.rating <- player.rating + K * (score - expected)
    LOG (delivery, d_rating, score, expected, player.rating)

# One control-loop tick, once per ball actually bowled:
record_outcome(player, delivery_just_faced, ball, reported_outcome)
next_delivery <- suggest_next_delivery(player, ball)
```

5. Demonstration: a synthetic 70-ball session

To check the loop actually behaves as intended, a synthetic batter with a hidden “true skill” (which also improves slowly over the session, the way a real player genuinely gets better with practice) faced 70 simulated balls, with outcomes drawn probabilistically from their true skill against whatever the algorithm chose to bowl.

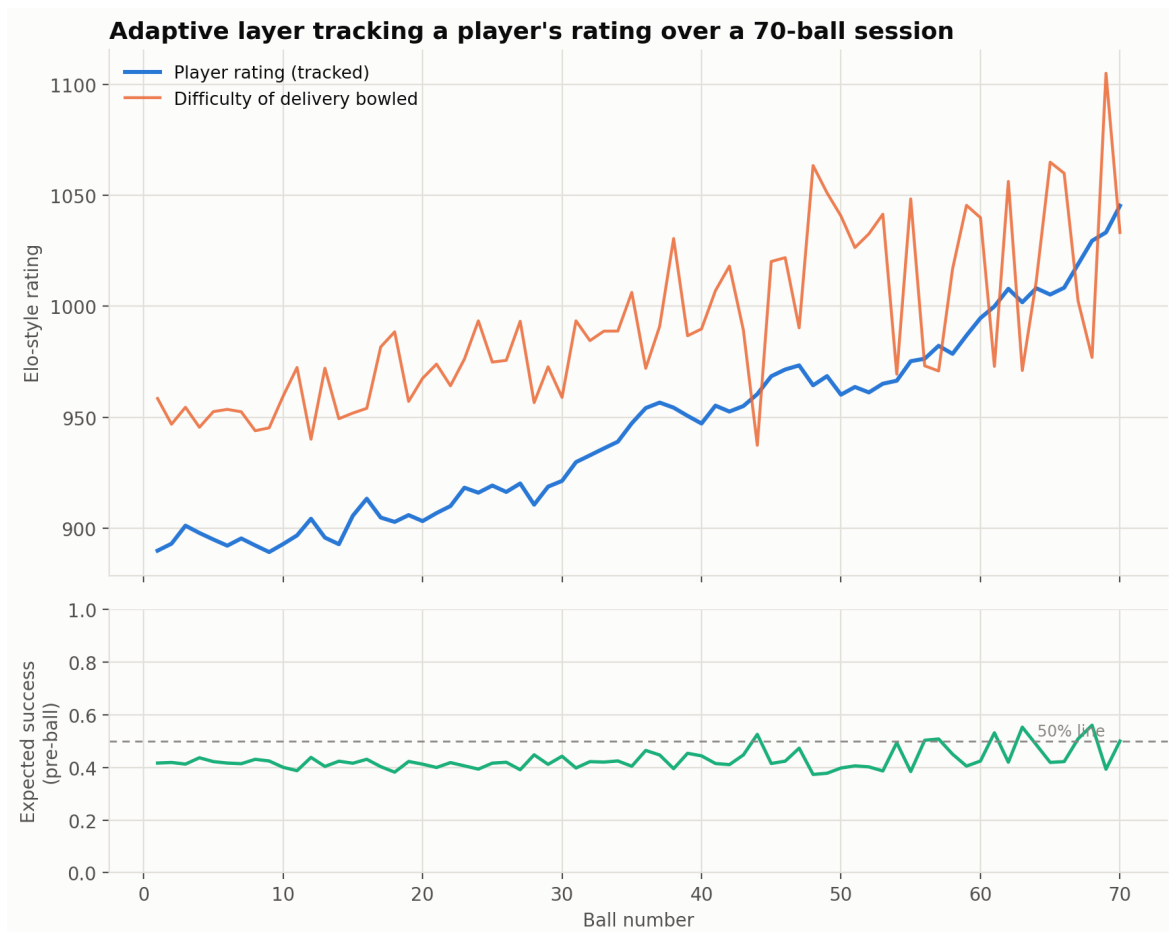


Figure 1. Top: the tracked player rating (blue) climbing as the synthetic batter improves, with the difficulty of each delivery bowled (orange) rising to stay just ahead of it. Bottom: expected pre-ball success probability, hovering just under 50% throughout — exactly the intended challenge band, rather than drifting toward “always easy” or “always impossible”.

Over this run the tracked rating climbed from 900 to about 1045, closely shadowing the batter's hidden true skill (which rose to about 1110), while the mean expected success across the whole session stayed at 0.43 — the machine kept finding deliveries in the right zone automatically, without anyone manually raising a difficulty slider.

6. How this plugs into the rest of the project

`cricket_trajectory.adaptive` exposes a small surface deliberately shaped to sit on top of the existing package rather than duplicate it:

- **PlayerProfile** — one object per batter: name, rating, K-factor, and a full history log. Persist this (e.g. to JSON) between sessions to carry progress forward.
- **delivery_difficulty_rating(delivery, ball)** — takes the same `Delivery/BallProperties` objects Section 4-5 of the main report already use.
- **suggest_next_delivery(profile, ball)** — returns a `Delivery` ready to hand straight to `WheelMachine.wheel_rpms_for_delivery()` from the machine-control module.
- **record_outcome()** / **next_delivery_after()** — the two calls a real control loop needs: log what happened, get what's next.

Nothing here depends on how the outcome is reported. Start with a coach tapping a button after each ball; swap in camera/sensor-based outcome detection later without touching the rating maths at all.

7. Limitations of this layer

- **Single skill dimension.** One Elo number can't separate “good against pace” from “struggles against spin” — a player who's excellent against seam and poor against wrist-spin gets one blended rating. A per-delivery-type rating set is a natural next step if that distinction matters.
- **Outcome scoring is subjective.** The 0-1 mapping in Section 2 is a starting point a coach should feel free to override for their own definition of “success”.
- **Cold start.** A brand-new player has no rating history, so the first several balls are effectively a rough guess until the rating settles.
- **No feedback sensor is built in.** As shipped this expects a human to report each outcome; it does not itself watch the shot and decide.
- **Elo assumes one match at a time.** It doesn't model fatigue, warm-up effects, or a bad ball being unluckily well-timed — those show up as short-term rating noise rather than being modelled explicitly.
- **The nastiness weighting (0.45/0.20/0.10/0.25) is a reasoned starting point, not a validated one** — exactly like the main report's aerodynamic curves, it deserves calibration against how real batters at a given level actually perform against each factor.